

Introduction to Machine Learning and Supervised Learning

Kritanta Saha

Assistant Professor
Dept. of Computer Science & Engineering
Sister Nivedita University

Jan, 2025

Outline

- ➊ Introduction to Machine Learning
- ➋ Types of Learning Techniques: Supervised, Unsupervised, Reinforcement Learning
- ➌ Regression Problem: Linear & Polynomial Regression
- ➍ Classification: Logistic Regression
- ➎ Algorithm for Regression & Classification: Gradient Descent

Introduction to Machine Learning

- **What is Machine Learning (ML)?**

It is a field of study that gives *computers the ability to learn* without being explicitly programmed. [Arthur Samuel 1959]

- **What is Learning?**

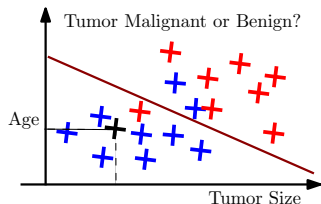
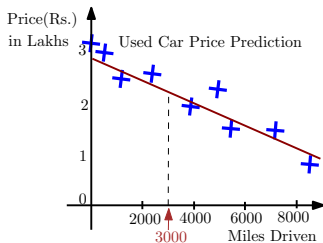
A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T as measured by P , improves with experience E . [Tom Mitchell 1998]

- **ML Applications:**

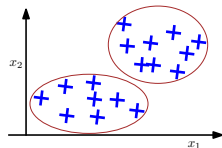
- Prediction: e.g. Used Car Price Prediction
- Classification: e.g. Detect a Tumor is Malignant or Benign?
- Clustering: e.g. User engagement in social media
- Association Rule Mining: e.g. Market Basket Analysis
- etc.

Different Types of Learning

- **Supervised Learning:** Given, a data set and known correct output i.e., having the idea that there is a relationship between the input and the output.

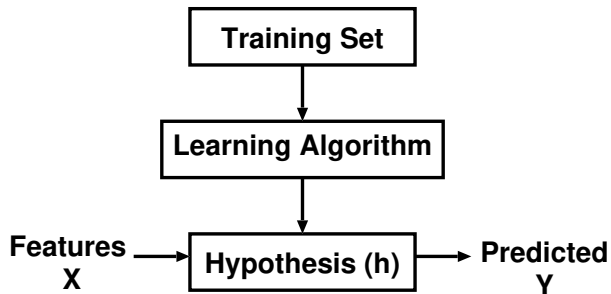


- **Unsupervised Learning:** It allows us to approach problems without any idea about what the results should look like.



- **Semi-supervised Learning (Reinforcement Learning):** It is a trial-and-error learning process where an agent interacts with an environment, takes actions, receives rewards, and improves its policy to maximize long-term rewards.

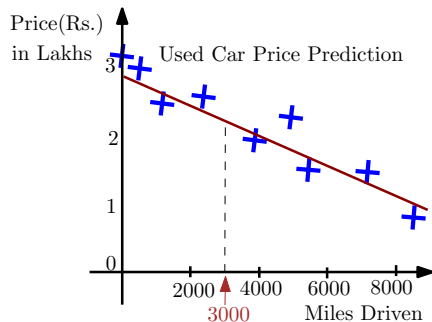
Overview of Supervised Learning



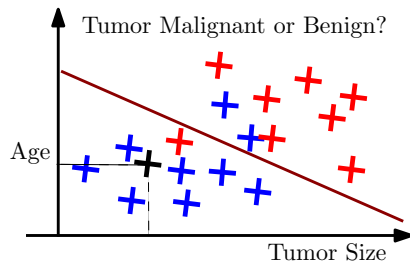
Overview of Supervised Learning

Supervised Learning

- **Regression Problem:** Maps input variables to some continuous function.
- e.g. Used Car Price Prediction
 - Feature: Miles Driven
 - Output: Price



- **Classification Problem:** Maps input variables into discrete categories.
- e.g. Detect whether tumor is malignant or benign?
 - Feature: Age and Tumor size
 - Output: Class label (Malignant or Benign)



Solving Regression Problem using Supervised Learning

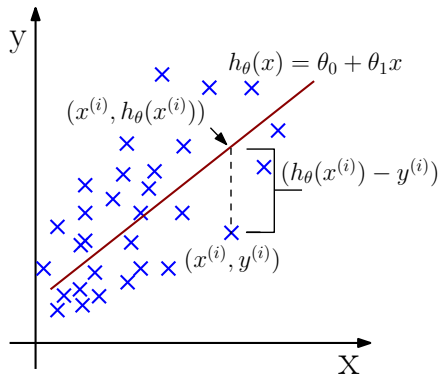
Consider Regression Problem: Used Car Price Prediction

- Training Dataset:

Miles Driven (x_1)	Engine Capacity(x_2) in hp	Price(y)
1230	1000	220000
3230	890	140500
..	..	..
..	..	..
4230	980	80000

- m : No. of Training examples
- x 's: Input variables or Features
- y 's: Output variables or targets
- (x, y) : One training example
- $(x^{(i)}, y^{(i)})$: i^{th} training example

Linear Regression with one variable/ Univariate Linear Regression



Univariate Linear Regression

- How to represent hypothesis h ?
$$h_{\theta} = \theta_0 + \theta_1 x$$
- Choose θ_0, θ_1 so that $h_{\theta}(x)$ is close to y for (x, y) .
- Error at i^{th} point = $(h_{\theta}(x^{(i)}) - y^{(i)}) \quad \forall i$.
- Mean Squared Error: $\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$
- Find θ_0, θ_1 , such that

$$\underset{\theta_0, \theta_1}{\text{Minimize}} \quad \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Linear Regression with One Variable

- Mean Squared Error Loss/Cost Function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

In summary, for Linear Regression with one variable

- Hypothesis : $h_{\theta} = \theta_0 + \theta_1 x$
- Parameters : θ_0, θ_1
- Loss/Cost Function :

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

- Goal :

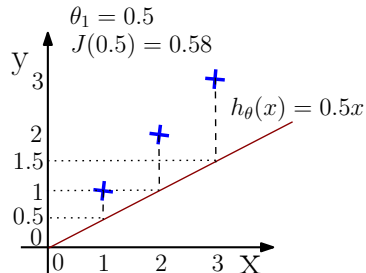
$$\underset{\theta_0, \theta_1}{\text{Minimize}} \quad J(\theta_0, \theta_1)$$

Understanding The Loss/Cost Function

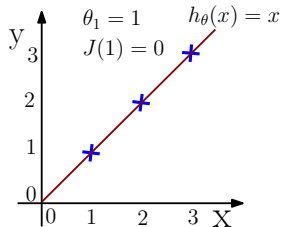
- Consider simplified hypothesis: $h_{\theta} = \theta_1 x$ [i.e., $\theta_0 = 0$]

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

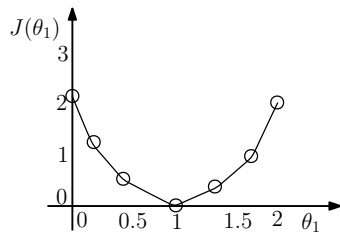
Minimize $J(\theta_1)$
 θ_1



$$J(0.5) = \frac{1}{2 \cdot 3} [(0.5 - 1)^2 + (1 - 2)^2 + (1.5 - 3)^2] = 0.58$$



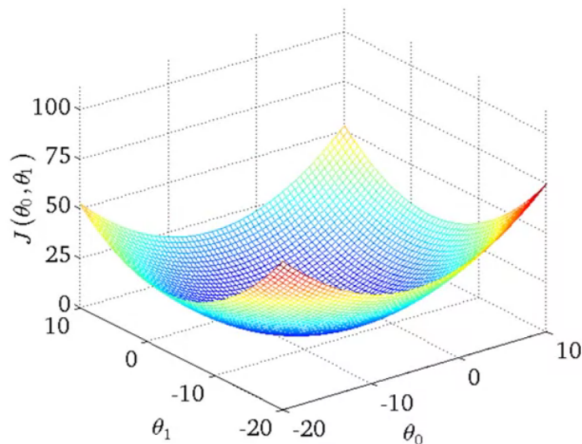
Varying θ_1



$\theta_1 = 1$ minimizes the cost function

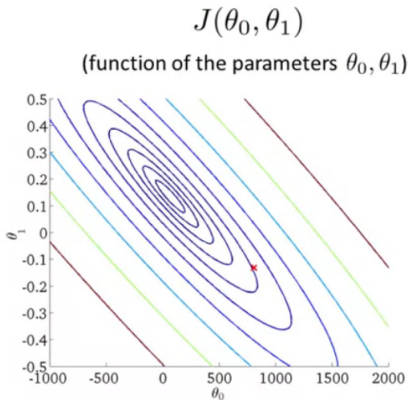
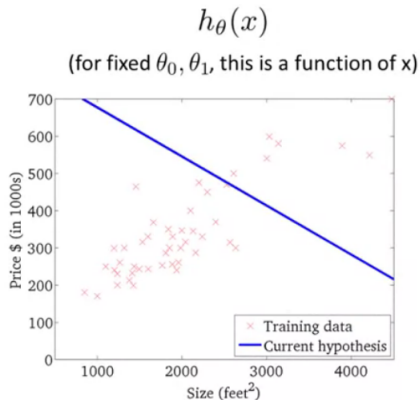
Understanding The Loss/Cost Function (cont..)

- Consider hypothesis $h_{\theta} = \theta_0 + \theta_1 x$



Loss/Cost function $J(\theta_0, \theta_1)$, Note that: The loss function is Convex

Understanding The Loss/Cost Function (cont..)



Linear Regression with Multiple variable

- n : Number of features
- $x^{(i)}$: Feature of i^{th} training example.
- $x_j^{(i)}$: Value of feature j in i^{th} training example.
- Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

- Parameters: $\theta_0, \theta_1, \dots, \theta_n$

$$X = \begin{bmatrix} x_0 \\ x_1 \\ . \\ . \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$\Theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ . \\ . \\ \theta_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$h_{\theta}(x) = \Theta^T X$$

$$J(\Theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

$$\underset{\Theta}{\text{Minimize}} \quad J(\Theta)$$

Polynomial Regression

- Fit a Quadratic Model i.e., Hypothesis:

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$

- Fit a Cubic Model i.e., Hypothesis :

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

- Fit other Models such as

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 \sqrt{x}$$

Algorithms for solving Linear Regression Problems

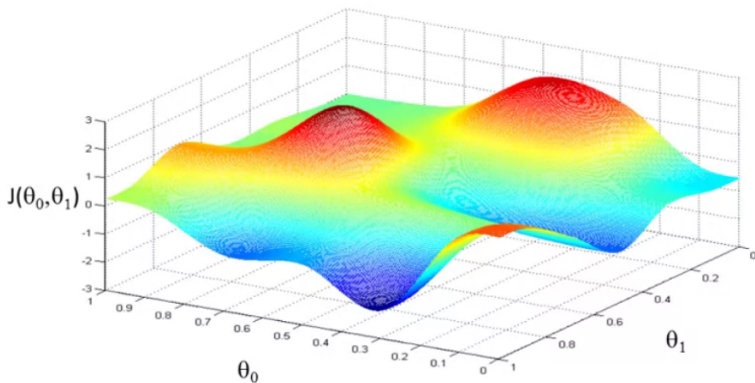
How to find the parameter vector Θ such that it minimizes Loss/cost function $J(\Theta)$?

To find the parameters that minimizes cost function $J(\Theta)$ use **Gradient Descent**

- Start with some random value of $\theta_0, \theta_1, \dots, \theta_n$. (Ex. say $\theta_i = 0 \forall i$).
- Keep changing $\theta_0, \theta_1, \dots, \theta_n$ to reduce $J(\theta_0, \theta_1, \dots, \theta_n)$ until we hopefully end up at a minimum.

Gradient Descent Algorithm

- Start with some value of $\theta_0, \theta_1, \dots, \theta_n$. (Ex. say $\theta_i = 0 \ \forall i$).
- Keep changing $\theta_0, \theta_1, \dots, \theta_n$ to reduce $J(\theta_0, \theta_1, \dots, \theta_n)$ until we hopefully end up at a minimum.



Intuition of Gradient Descent

Gradient Descent Algorithm

Gradient Descent for Solving Linear Regression with One Variable

Repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \quad (\text{for } j = 0 \text{ and } j = 1)$$

}
where α is the Learning Rate.

Correct Simultaneous Update

$$temp0 := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

$$temp1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

$$\theta_0 := temp0$$

$$\theta_1 := temp1$$

Gradient Descent for Univariate Linear Regression

$$\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \left(\frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 \right) \quad (\text{for } j = 0 \text{ and } j = 1)$$

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \quad \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

Repeat until convergence {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x^{(i)}$$

}
where α is the Learning Rate.

Gradient Descent For Multivariate Linear Regression

Repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\Theta) \quad (\text{simultaneously update for every } j = 0, 1, \dots, n)$$

}
where α is the Learning Rate.

$$J(\Theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

$$\frac{\partial}{\partial \theta_j} J(\Theta) = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

Effect of Learning Rate α

- α too small $\implies$ Gradient descent can be slow.
- α too large $\implies$ Gradient descent can overshoot the minimum.
Fail to converge or even diverge.
- For a Fixed α , Gradient descent can converge to local minimum.
- **How to make sure gradient descent is working correctly?**
 - If Gradient descent works properly then $J(\Theta)$ should decrease after every iteration.
- **How to choose α ?**
 - If Gradient descent increases then use smaller α .

Feature Scaling

- Idea: Make sure features are on a similar scale.
- Feature Scaling to eliminate the need for extra steps to reach at local optimal.
- For example:

Let, x_1 = Size of House (0 - 2000 *feet*²)

and x_2 = Number of bedrooms (1-5)

$$x_1 = \frac{\text{Size of house}}{2000} \text{ and } x_2 = \frac{\text{No. of Bedrooms}}{5}$$

Mean Normalization Feature Scaling Technique

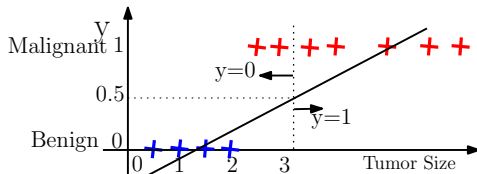
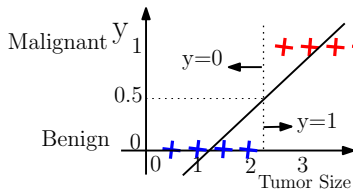
- Replace x_i with $x_i - \mu_i$ (Do not apply to x_0)
- Ex. $x_1 = \frac{\text{Size of house} - 1000}{2000}$ and $x_2 = \frac{\text{No. of Bedrooms} - 2}{5}$

Revisit Supervised Learning

- Regression Problem
- **Classification Problem**

Classification Problem

- Binary Classification problem: $y \in \{0, 1\}$
where 0 : Negative class 1 : Positive class.
- Multiclass Classification problem: $y \in \{0, 1, 2, 3\}$ etc.
- Ex. Email Spam/Not Spam?
Tumor malignant/Benign?
- Linear regression may not work and wrongly classify.



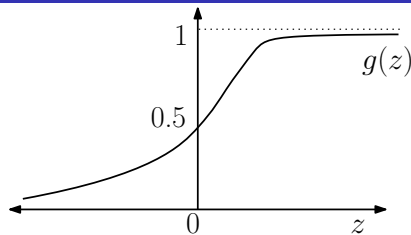
Logistic Regression

Hypothesis Representation

- Required : $0 \leq h_{\theta}(x) \leq 1$

$g(z)$: Sigmoid or Logistic function

$$g(z) = \frac{1}{1 + e^{-z}}$$
$$h_{\theta}(x) = g(\Theta^T x) = \frac{1}{1 + e^{-\Theta^T x}}$$



Sigmoid or Logistic Function

Interpretation of Hypothesis

$h_{\theta}(x) =$ *Estimated Probability that $y = 1$ on input x .*

$$\text{i.e., } h_{\theta}(x) = P(y = 1|x; \theta)$$

Example

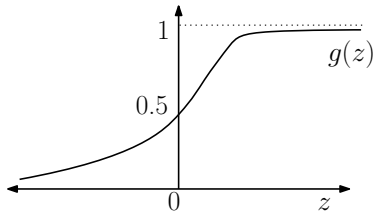
$$x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{Tumorsize} \end{bmatrix}$$

$$h_{\theta}(x) = 0.7$$

$\implies$ 70% chance of tumor being Malignant.

Logistic Regression: Decision Boundary

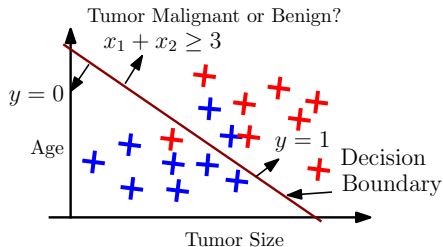
- $h_{\theta}(x) = g(\Theta^T X)$
- Predict
 $y = 1$ if $h_{\theta}(x) \geq 0.5$
 $y = 0$ if $h_{\theta}(x) < 0.5$



- $g(\Theta^T X) \geq 0.5$ when $\Theta^T X \geq 0$
 $\implies y = 1$ if $\Theta^T X \geq 0$
 $y = 0$ if $\Theta^T X < 0$

Example

- $h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$
- Let $\theta_0 = -3$, $\theta_1 = 1$ and $\theta_2 = 1$
- Predict
 $y = 1$ if $-3 + x_1 + x_2 \geq 0$
 $y = 0$ if $x_1 + x_2 < 3$



Logistic Regression: Cost Function

- m training examples $(x^{(1)}, y^{(1)}) , (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$

- n features $x = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1}$ where $x_0 = 1$ and $y \in \{0, 1\}$

- Hypothesis :

$$h_{\theta}(x) = \frac{1}{1 + e^{-\Theta^T x}}$$

- Cost Function for Linear Regression

$$J(\Theta) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

$$Cost \left(h_{\theta}(x^{(i)}), y^{(i)} \right) = \frac{1}{2} \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Logistic Regression: Cost Function

Binary Classification Problem

- Cost Function for Logistic Regression

$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

$$J(\Theta) = \frac{1}{m} \sum_{i=1}^m Cost(h_{\theta}(x^{(i)}), y^{(i)})$$

$$J(\Theta) = -\frac{1}{m} \left(\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right)$$

- To fit parameter θ

$$\underset{\Theta}{\text{Minimize}} \quad J(\Theta)$$

Gradient Descent for Logistic Regression

$$\frac{\partial}{\partial \theta_j} J(\Theta) = \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

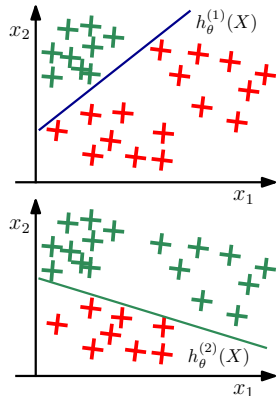
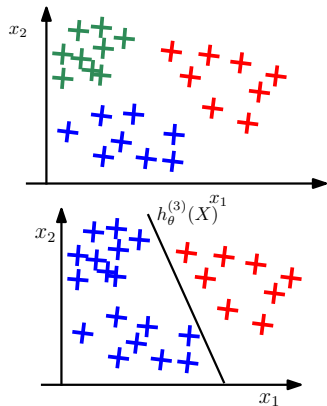
Repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\Theta) \quad (\text{simultaneously update for every } j = 0, 1, \dots, n)$$

}
where α is the Learning Rate.

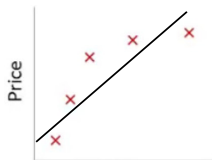
- Faster ways to optimize : Conjugate gradient, BFGS, and L-BFGS.

Logistic Regression : Multiclass Classification

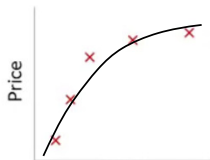


- $h_{\theta}^{(i)}(x) = P(y = i|x; \theta)$ for $i = 1, 2, 3$, train $h_{\theta}^{(i)}(x)$ for each class i to predict the probability that $y = i$.
- On a new input x , to make a prediction pick the class i that maximizes $h_{\theta}^{(i)}(x)$

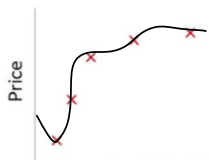
Regularization : Overfitting Problem



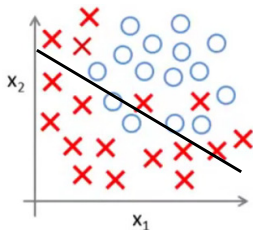
Size
 $\theta_0 + \theta_1 x$



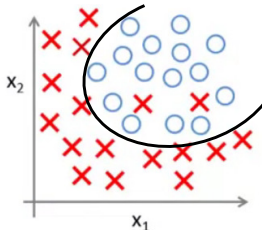
Size
 $\theta_0 + \theta_1 x + \theta_2 x^2$



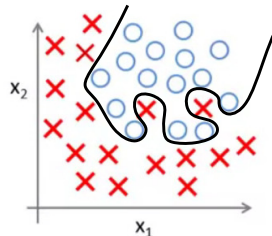
Size
 $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$



$\hat{y}_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$
(g = sigmoid function)



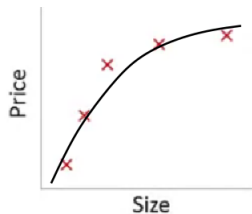
$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$



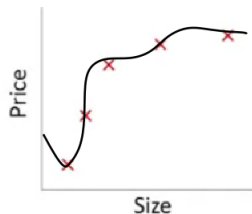
$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \dots)$

Regularization : Solving Overfitting Problem

- Reduce No. of Features
- Regularization
 - Keep all the features but reduce magnitude of parameter θ_j



$$\theta_0 + \theta_1 x + \theta_2 x^2$$



$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

- For Example,

$$\text{Min}_{\theta} \quad \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + 1000\theta_3 + 1000\theta_4$$

Regularization for Linear Regression

$$J(\Theta) = \frac{1}{2m} \left(\sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^n \theta_j^2 \right)$$

where λ is the regularization parameter.

Gradient Descent With Regularization

Repeat until convergence {

$$\begin{aligned} \theta_0 &:= \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_0^{(i)} \\ \theta_j &:= \theta_j - \alpha \left(\left(\frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)} \right) + \frac{\lambda}{m} \theta_j \right) \quad j \in \{1, 2..n\} \end{aligned}$$

}

where α is the Learning Rate and λ is the regularization parameter.

Regularization for Logistic Regression

$$J(\Theta) = -\frac{1}{m} \left(\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right) + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

where λ is the regularization parameter.

Gradient Descent With Regularization

Repeat until convergence {

$$\begin{aligned} \theta_0 &:= \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_0^{(i)} \\ \theta_j &:= \theta_j - \alpha \left(\left(\frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)} \right) + \frac{\lambda}{m} \theta_j \right) \quad j \in \{1, 2..n\} \end{aligned}$$

}

where α is the Learning Rate and λ is the regularization parameter.

- Book 1: Ethem Alpaydin, “Introduction to Machine Learning”, 3rd Edition.
- Book 2: Tom M. Mitchell, “Machine Learning” , McGraw-Hill
- Online: Machine Learning course by Stanford University.